

Functions & Events

reusable blocks of code, and reacting to the user

1 What is a function?

A **function** is a reusable block of code that does one job. You write it once, then run it whenever you need it, as many times as you like.

```
// define once
function greet() {
  console.log("Hello!");
}
// run it any time (this is "calling")
greet();
greet();
```

2 Why do we use functions?

- **Reusability** - write once, use many times
- **Maintainability** - fix a bug in one place, not ten
- **Organised code** - each job lives in its own box
- **Easier debugging** - problems are easy to locate
- **Better readability** - a good name explains what it does
- **Unit testing** - small pieces are easy to test on their own

3 What makes a good function?

- **One job** (single responsibility) - it does one thing well
- keep it **short**, around 20 to 30 lines is a good target
- if it grows too big, **break it into smaller functions**

Rule of thumb: if you cannot describe what a function does in one short sentence, it is probably doing too much. Split it.

4 Syntax: parts of a function

```
function add(a, b) { // a, b = parameters
  return a + b;      // return = send a value back
}
let total = add(2, 3); // 2, 3 = arguments -> total is 5
```

parameters vs arguments

- **Parameters** - the names in the definition (a, b). The empty slots.
- **Arguments** - the real values you pass in when calling (2, 3).

Easy way to remember: Parameters are the Placeholders. Arguments are the Actual values.

calling (invoking) a function

Writing the function does nothing on its own. You have to **call** it with its name and brackets: **add(2, 3)**. That is when the code inside actually runs.

5 The return statement

return sends a value back out of the function so you can use it. Once return runs, the function **stops** right there.

```
function square(n) {
  return n * n;
  console.log("never runs"); // dead code after return
}
```

Functions without return: they just do something (like print to the screen) and hand back **undefined**. That is fine when you do not need a value back.

function vs procedure

In older languages there was a clear split:

- a **function** takes input and **returns a value** (like add(2,3) giving back 5)
- a **procedure** just **does a job** and returns nothing (like printing to the screen)

In JavaScript: we call both of them \u201Cfunctions.\u201D There is no separate procedure keyword. A function that returns a value and one that just does a job are written the same way, one simply has a return and the other does not.

6 Pure vs Impure functions

pure

- same input **always** gives the same output
- changes nothing outside itself (no side effects)
- easy to test and trust

```
function add(a,b){  
  return a+b;  
}
```

impure

- output can change even with the same input
- touches the outside world (DOM, a global, random, time)

```
let count=0;  
function tick(){  
  count++;      // changes  
outside  
}
```

Aim for pure where you can. Pure functions are predictable, which makes bugs rare and testing simple.

7 Three ways to write a function

1. function declaration

```
function greet(name) {  
  return "Hi " + name;  
}
```

2. function expression

```
// stored in a variable  
const greet = function(name) {  
  return "Hi " + name;  
};
```

3. arrow function

```
const greet = (name) => {  
  return "Hi " + name;  
};
```

8 Arrow function variations

Arrow functions are just a shorter way to write a function. They have a few handy shortcuts:

implicit return (one line, no braces)

If the body is a single expression, drop the braces and the word `return`. The value comes back on its own.

```
const add = (a, b) => a + b;    // returns a+b
```

single parameter (brackets optional)

```
const square = n => n * n;      // no () needed for one param
```

multiple parameters (brackets needed)

```
const add = (a, b) => a + b;    // () needed for 2+ params
```

Quick guide: one line of logic? use [implicit return](#). One parameter? you can [drop the \(\)](#). Zero or 2+ parameters? [keep the \(\)](#).

same function, written 5 ways (all equal)

These all do the exact same thing. Read them top to bottom, each one is a little shorter than the last:

```
// 1. declaration
function add(a, b) { return a + b; }

// 2. expression (function stored in a variable)
const add = function(a, b) { return a + b; };

// 3. arrow, full body
const add = (a, b) => { return a + b; };

// 4. arrow, implicit return
const add = (a, b) => a + b;

// 5. single param uses _ when unused
const add = _ => a + b;
```

The `_` trick: `_` is just a parameter name people use to mean "I am not using this argument." It is not special syntax, any name would work, but `_` signals "ignore me."

style	looks like
declaration	<code>function add(a,b){ return a+b; }</code>
expression	<code>const add = function(a,b){...};</code>
arrow (full)	<code>const add = (a,b) => { return a+b; };</code>
arrow (short)	<code>const add = (a,b) => a+b;</code>

making the page react

JS Events

Events

reacting to what the user does

9 What is an event?

An **event** is something that happens on the page: a click, a key press, the mouse moving, a form being submitted. You attach a function that runs when that thing happens. This is what makes a page interactive.

10 addEventListener

The main tool. You give it two things: the **event name** to listen for, and the **function** to run when it fires.

```
btn.addEventListener("click", () => {  
  console.log("clicked!");  
});
```

The function you pass in is called a **callback** - the browser calls it back for you when the event happens. You can pass a named function too:

```
function changeColor() { ... }  
btn.addEventListener("mouseout", changeColor);
```

Watch out: pass the function name without brackets (`changeColor`), not `changeColor()`. Brackets would run it immediately instead of waiting for the event.

11 The event object

When an event fires, the browser hands your function an **event object** full of details, like which element was involved and where the mouse was.

```
btn.addEventListener("click", (event) => {  
  console.log(event.target); // the element clicked  
  console.log(event.type);   // "click"  
});
```

12 Common events

event	fires when...
click	an element is clicked
dblclick	an element is double-clicked
mousemove	the mouse moves over an element
mouseover	the mouse enters an element
mouseout	the mouse leaves an element
mousedown	a mouse button is pressed down
keydown	a key is pressed down
keyup	a key is released
input	the value of a field changes as you type
change	a field changes and loses focus
submit	a form is submitted
blur	an element loses focus
scroll	the user scrolls
resize	the window is resized
load	the page (or an image) finishes loading

13 Handling events: the pattern

Three steps, every time:

- **Select** the element with `querySelector`
- **Listen** with `addEventListener`

→ **Do** something inside the callback (change the DOM)

```
// 1. select
const btn = document.querySelector(".btn");

// 2. write the handler
function changeBackgroundColor() {
  document.body.style.background = "lightblue";
}

// 3. listen, and pass the handler as a callback
btn.addEventListener("mouseout", changeBackgroundColor);
```

Select, listen, do. Almost every interactive feature on the web is just this loop with a different event and a different action.

14 Removing an event listener

You can take a listener off again with **removeEventListener**. You pass the same event name and the same function you added.

```
btn.addEventListener("click", changeColor);
// later, stop listening
btn.removeEventListener("click", changeColor);
```

why remove one?

- **One-time actions** - a handler that should run once, then stop (e.g. a "claim reward" button)
- **Cleanup** - when an element goes away, remove its listeners so they do not pile up and waste memory
- **Turning behaviour off** - stop reacting once something is done or disabled

Important: it only works if you pass the same named function. An inline arrow like `() => {...}` is a brand new function each time, so the browser cannot find it to remove it. Use a named function when you plan to remove it.

15 What if JavaScript is off?

JavaScript can be switched off, or fail to load. So a page should still make sense without it. This idea is called **graceful degradation**: the core content and links work with plain HTML, and JS only adds extra polish on top.

- write real HTML content and real links first
- use JS to **enhance**, not to hold the whole page together
- if JS is the only thing making the page usable, users with it off see a broken page

turning JS off yourself (a testing skill)

You can disable JavaScript in the browser to test how your page behaves without it: open DevTools (F12), go to Settings or the command menu, and toggle \u201CDisable JavaScript.\u201D Reload and see what still works.

Try it: turn JS off and open your own page. If the content still reads and the links still work, you built it well. If it is blank, the page leans on JS too heavily.

class record

Minutes

Class MOM

what we did in today's session

Module	Module 1 - JavaScript
Topics	Functions, and Events
Instructor	Dinesh Rawat

what we covered

- **Functions** - what they are, why we use them, and what makes a good one (one job, kept short, split when big)
- **Syntax** - parameters vs arguments, calling a function, the return statement, and functions with no return
- **Function vs procedure** - the classic split, and how JavaScript just calls both \u201Cfunctions\u201D
- **Pure vs impure** functions, and why pure ones are easier to trust and test
- **Three ways to write a function** - declaration, expression, and arrow, plus the arrow shortcuts and the \u201Csame function five ways\u201D chain
- **Events** - what they are, addEventListener, callbacks, and the event object

- the **common events** (click, mouseover, keydown, input, submit and the rest)
- the **select - listen - do** pattern for handling events
- **removeEventListener** - how and why to take a listener off
- **graceful degradation** - keeping a page usable when JavaScript is off

key takeaways

- ★ a function should do one thing, and its name should tell you what
- ★ parameters are placeholders, arguments are the real values
- ★ pass a callback by name (no brackets) to addEventListener
- ★ to remove a listener you need the same named function
- ★ every interactive feature = select, listen, do

next up

We put functions and events together to build real interactive pages.

